

Report on Sorting Algorithms in Practice

Introduction. This study investigates the performance of three sorting algorithms (Merge Sort, Quick Sort - Lomuto partition, and Quick Sort - Hoare partition). runtime was recorded in microseconds for all three algorithms across 100 randomly sized user friends lists. In this study we want to determine whether there is a significant difference in their theoretical time complexities vs in practice, and how do they compare to each other.

Table: Sorting Algorithms Summary Statistics

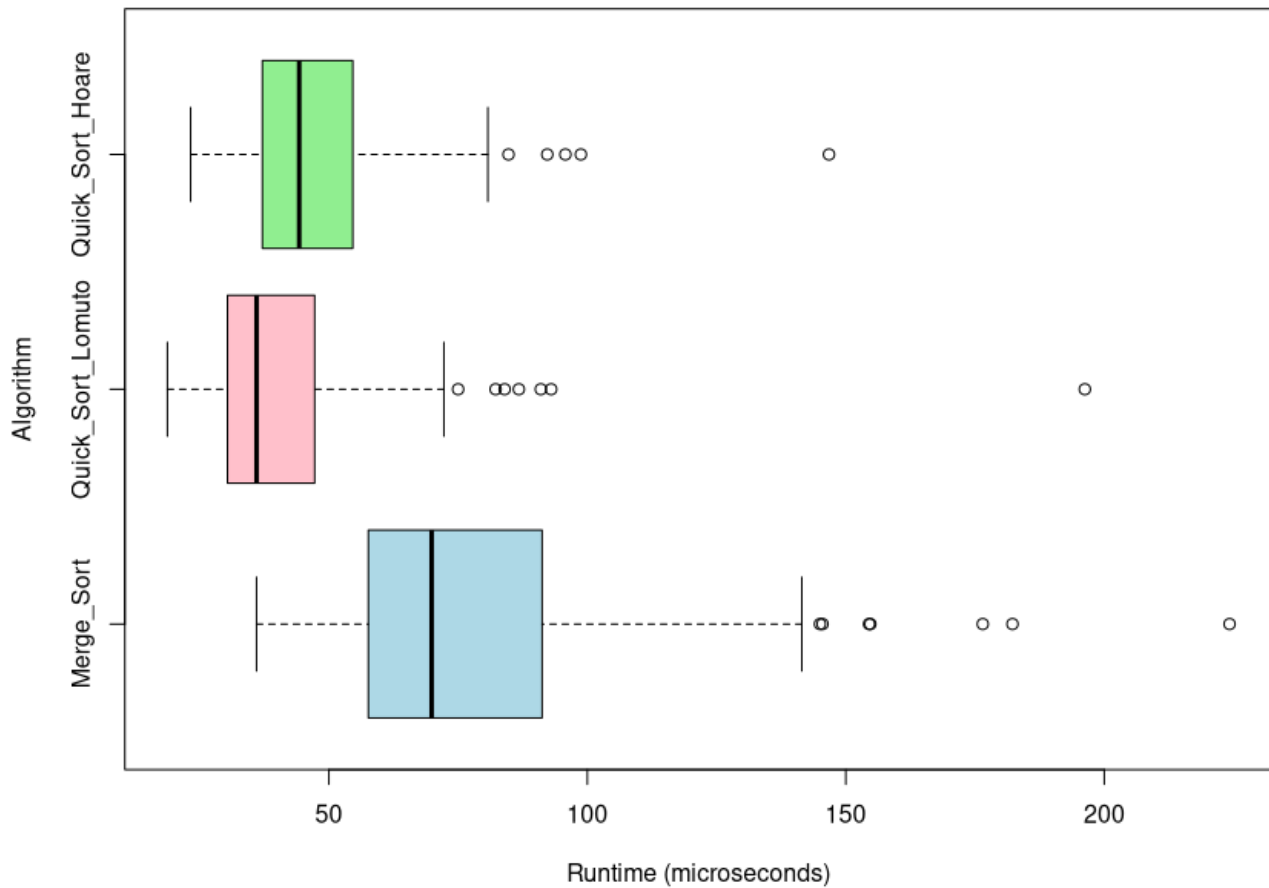
Sorting Algorithm	Merge Sort	Quick Sort Lomuto	Quick Sort Hoare
Min	36.00	18.75	23.25
Q1	57.69	30.44	37.19
Median	69.88	36.00	44.25
Q3	90.88	47.12	54.19
Max	224.25	196.25	146.75
Mean	79.61	42.47	48.07
Standard Deviation	34.15	22.59	18.21

The summary statistics tells us that between the algorithms average-case runtime and worst case. On average Quick Sort using Lomuto (QSL) partition is the fastest algorithm for this dataset, having the lowest mean (42.47) runtime. Quick Sort using Hoare (QSH) partition was close behind with a mean of (48.07). While Merge Sort performance was significantly slower across all cases, with a mean of (79.61).

While QSL has a faster average case runtime the table also shows that its worst-case runtime is vulnerable to edge-case data. With it having such a massive spike (196.25) and a higher Standard Deviation then QSH shows that it QSL is more unstable than QSH. Also, the large gap between median (36.00) and mean (42.47) shows that its average is being skewed higher by the worst-case outliers. While QSH is more stable with its median (44.25) and mean (48.07) being closer showing it's not being skewed by some outliers as much. While Merge Sort having a theoretical better worst-case runtime, has the highest Q3 (90.88) and overall worst in every case while also being the most unstable with the biggest standard deviation (34.15)

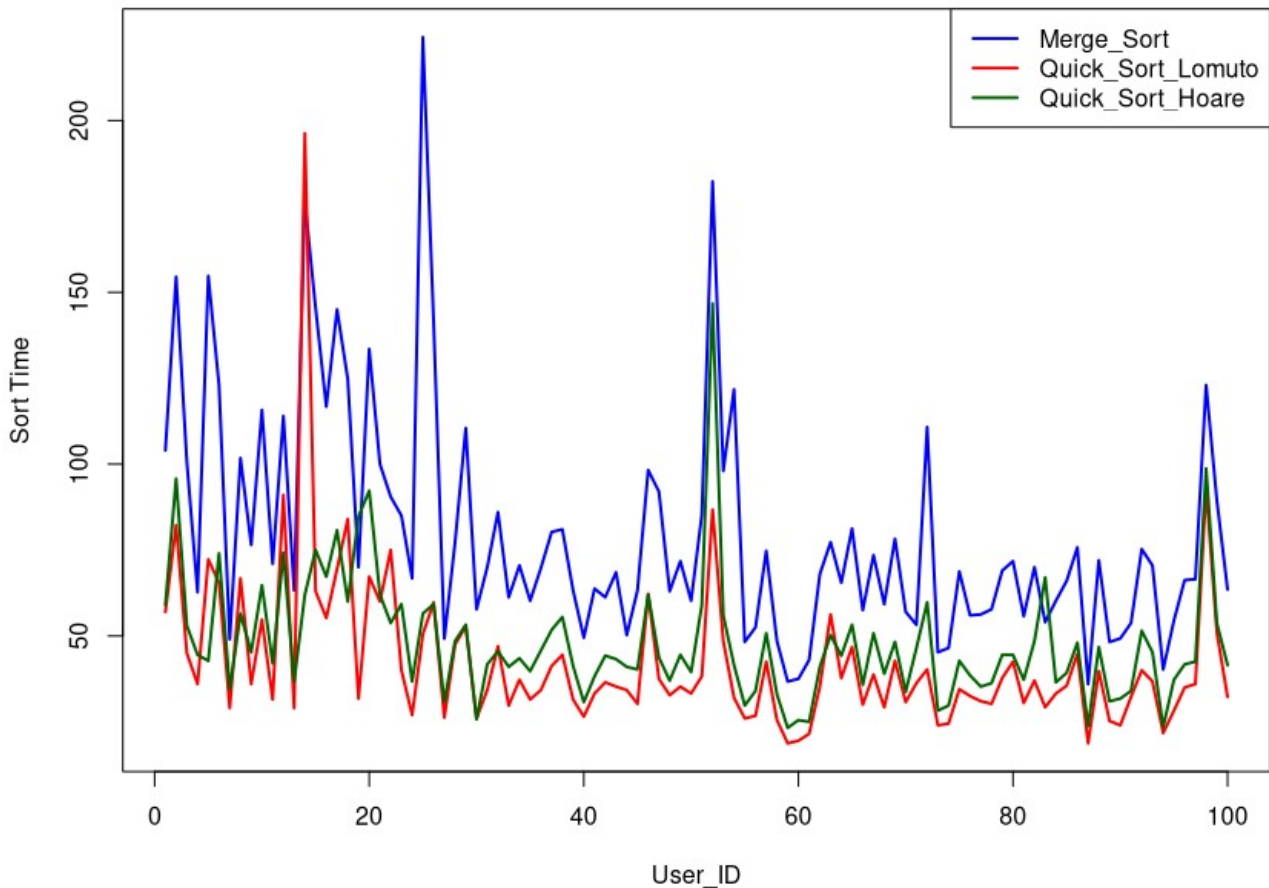
We can also visually see the instability with each Sorting Algorithm with a box plot. With QSL we can confirm that the majority of the times were fastest out of the algorithms, but QSL has more outliers (outside the whiskers) showing its unstable.

Distribution of Sorting Runtime



We can also look at a line graph of the 100 users sort times to see that Merge Sort (blue) is hovering over the other lines showing that on average will run slower than the Quick Sorts. We can also see a big peak for QSL (red) in comparisons to everything else runtime, show its volatility to edge cases.

Sorting Algorithm On User 100



To validate our data from the summary statistics, we conducted two paired t-tests using standard significance level of 0.05.

The first test was compared the overall Merge Sort and QSL. The test shows with a p-value of $2.2e-16$. With the p-value being so low and much smaller than 0.05. This statistically proves that QSL runtime speed over Merge Sort is true for this dataset.

The second test was comparing QSH and QSL to see that QSL slight runtime speed was statistically significant. The test shows with a p-value of 0.0022. With the p-value being lower than 0.05, we can confirm that QSL is faster on average than QSH for this dataset.

Conclusion: Theoretically, Merge Sort should guarantee a time complexity of $O(n \log n)$ in all cases, which should make it the safest choice over Quick Sort with $O(n^2)$ at worst-case. But, in practice the results of this test shows highly significance evidence that the Quick Sort options are a much safer choice. With QSL being the fastest sorting algorithm on average while QSH being the most stable.

Mostly likely the hardware for memory management is hurting Merge Sort a lot more than the Quick Sorts. With Merge Sort relying on so much memory to allocate. So, for smaller scale Merge Sort overhead of managing memory takes to much time over its time it saves from being more efficient algorithm.